LAB SHEET — EXPERIMENT 4

Experiment Name: Frequency Domain Image Filtering using Fourier Transform

Aim

To implement image filtering in the frequency domain using the Fourier Transform and analyze the effectiveness of frequency-based filtering techniques for noise removal, image enhancement, and feature preservation using Python and OpenCV.

Purpose

The Fourier Transform represents an image in terms of its spatial frequencies. This allows low-frequency and high-frequency components to be manipulated separately. Low-pass filtering retains gradual intensity variations and suppresses rapid changes, while high-pass filtering suppresses gradual variations and emphasizes edges and fine details.

Prerequisites

• Basic digital image processing and Python programming
• Understanding of spatial and frequency domains
• NumPy, OpenCV (cv2), and Matplotlib
• Jupyter Notebook, Google Colab, or Visual Studio Code



Workflow: image → Fourier Transform → frequency mask → inverse transform → compare results.

1. Import Libraries and Prepare an Image

The question sheet does not provide a specific input image. For a reproducible demonstration, the following code creates a synthetic grayscale test image and adds Gaussian noise. Replace this generated image with your own grayscale image if required.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a reproducible grayscale test image (256 x 256)
np.random.seed(7)
image = np.zeros((256, 256), dtype=np.uint8) + 35
cv2.rectangle(image, (24, 24), (110, 100), 170, -1)
cv2.circle(image, (177, 75), 38, 220, -1)
cv2.line(image, (25, 155), (225, 155), 245, 4)
cv2.rectangle(image, (45, 180), (100, 230), 110, 3)
cv2.putText(image, "DFT", (130, 205),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, 220, 2)

noise = np.random.normal(0, 18, image.shape)
noisy_image = np.clip(image.astype(float) + noise, 0, 255).astype(np.uint8)
print("Image shape:", image.shape)
print("Image dtype:", image.dtype)
print("Noisy image mean:", round(noisy_image.mean(), 2))

plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1); plt.imshow(image, cmap="gray"); plt.title("Original")
plt.subplot(1, 2, 2); plt.imshow(noisy_image, cmap="gray"); plt.title("Noisy")
plt.tight_layout(); plt.show()
```

Output

Image shape: (256, 256) Image dtype: uint8
Noisy image mean: 66.90

Original grayscale image

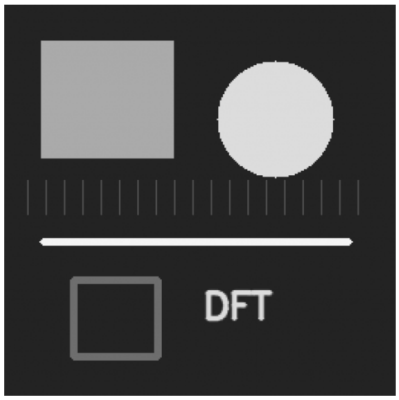
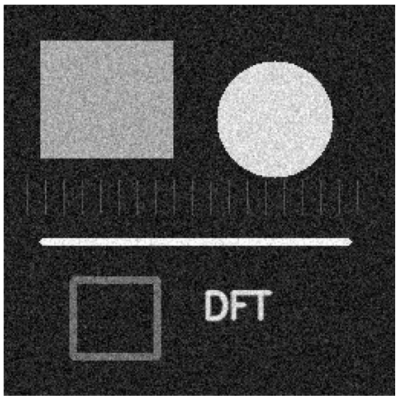
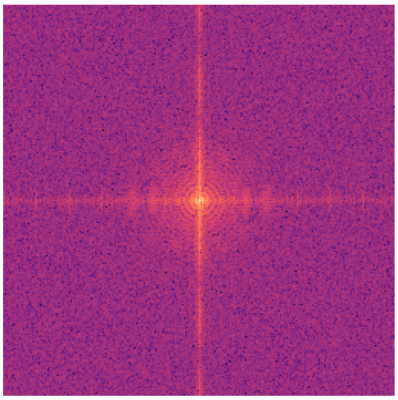


Image with Gaussian noise



Centered Fourier magnitude spectrum



2. Compute the Discrete Fourier Transform

The 2-D DFT converts the image into complex frequency coefficients. The shift operation moves the zero-frequency (DC) component to the center so the spectrum is easier to interpret. The log magnitude makes weak frequency components visible.

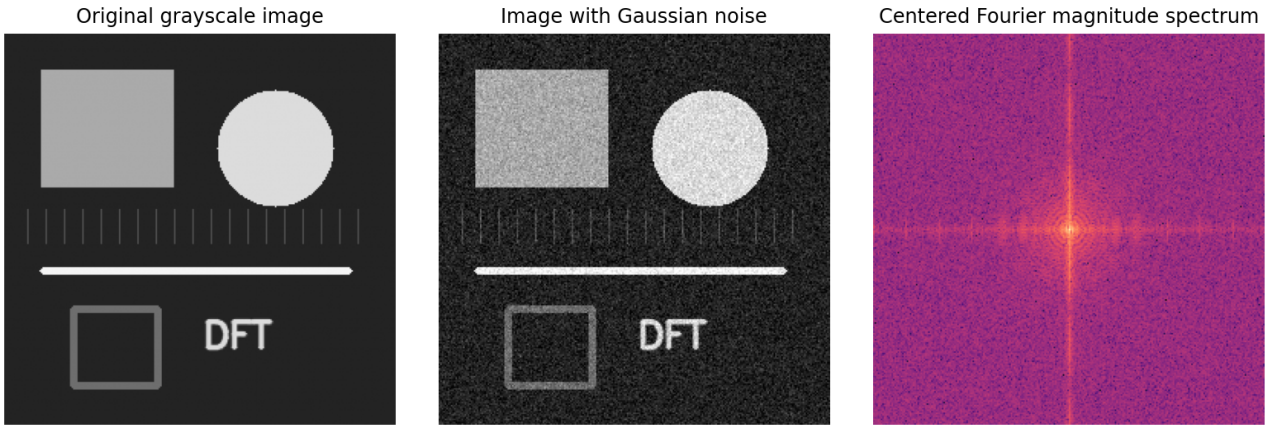
```
# Compute the 2-D Fourier Transform and center the spectrum
f_transform = np.fft.fft2(noisy_image)
f_shift = np.fft.fftshift(f_transform)
magnitude_spectrum = 20 * np.log(np.abs(f_shift) + 1)

print("DFT array shape:", f_transform.shape)
print("Magnitude spectrum shape:", magnitude_spectrum.shape)

plt.figure(figsize=(5, 4))
plt.imshow(magnitude_spectrum, cmap="magma")
plt.title("Centered Fourier Magnitude Spectrum")
plt.axis("off")
plt.colorbar(label="Log magnitude")
plt.tight_layout(); plt.show()
```

Output

DFT array shape: (256, 256)
Magnitude spectrum shape: (256, 256)



3. Apply a Low-Pass Frequency Filter

A circular low-pass mask retains frequency components close to the spectrum center and removes components outside the selected radius. This smooths the image and can reduce high-frequency noise, but may blur edges and small details.

```
rows, cols = noisy_image.shape
crow, ccol = rows // 2, cols // 2
y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol)**2 + (y - crow)**2)

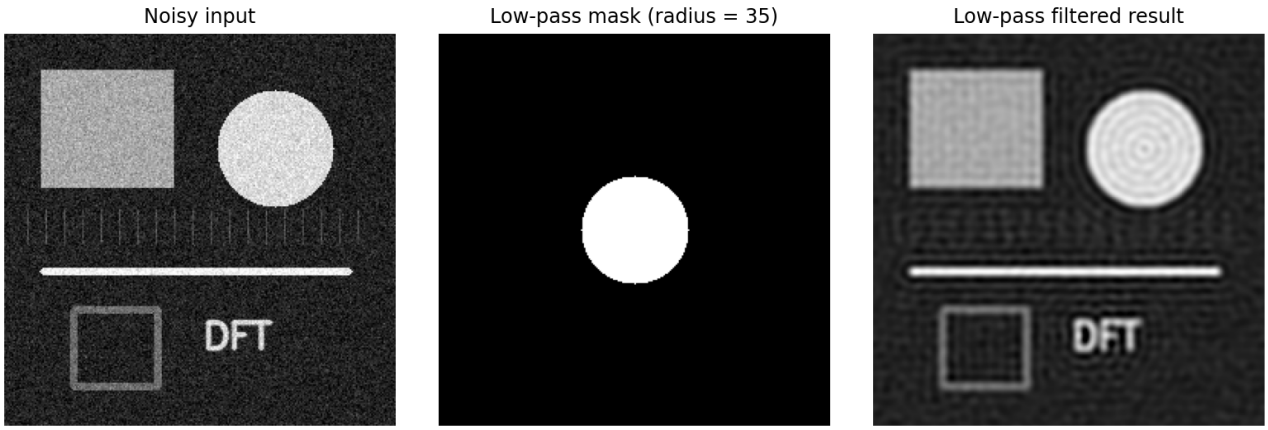
# Keep frequencies within a radius of 35 pixels
low_pass_mask = (distance <= 35).astype(np.uint8)
filtered_low_freq = f_shift * low_pass_mask
low_pass_result = np.fft.ifft2(np.fft.ifftshift(filtered_low_freq))
low_pass_result = np.real(low_pass_result)
low_pass_result = np.clip(low_pass_result, 0, 255).astype(np.uint8)

print("Low-pass mask values:", np.unique(low_pass_mask))
print("Low-pass result shape:", low_pass_result.shape)

plt.figure(figsize=(9, 3))
plt.subplot(1, 3, 1); plt.imshow(noisy_image, cmap="gray"); plt.title("Noisy input")
plt.subplot(1, 3, 2); plt.imshow(low_pass_mask, cmap="gray"); plt.title("Low-pass mask")
plt.subplot(1, 3, 3); plt.imshow(low_pass_result, cmap="gray"); plt.title("Low-pass result")
for a in plt.gcf().axes[:3]: a.axis("off")
plt.tight_layout(); plt.show()
```

Output

Low-pass mask values: [0 1]
Low-pass result shape: (256, 256)



4. Apply a High-Pass Frequency Filter

A high-pass mask suppresses frequencies near the center and retains components farther away. These components often correspond to edges, fine structures, and noise. The displayed high-pass response is normalized for visualization; it is not intended to preserve the original brightness.

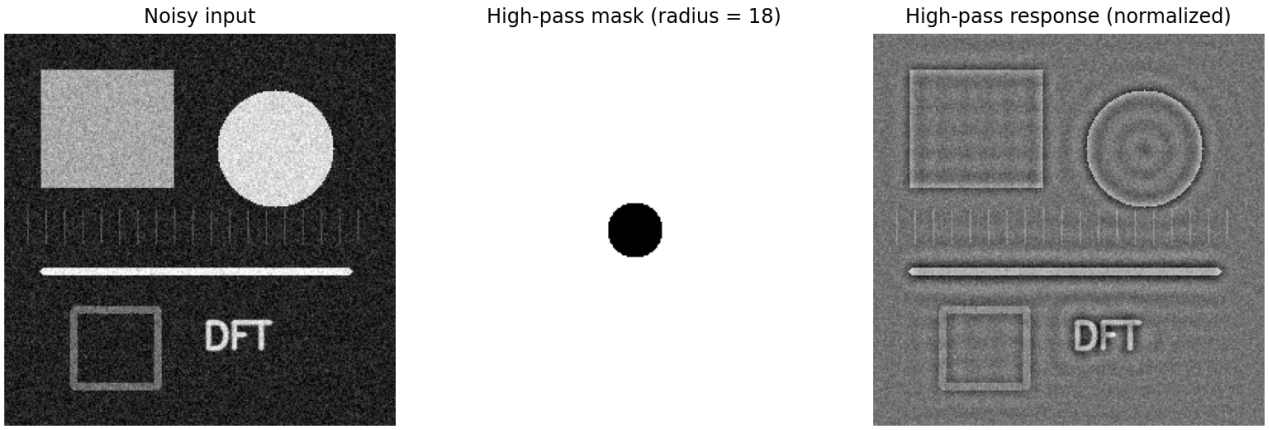
```
# Reject frequencies within a radius of 18 pixels
high_pass_mask = (distance >= 18).astype(np.uint8)
filtered_high_freq = f_shift * high_pass_mask
high_pass_result = np.fft.ifft2(np.fft.ifftshift(filtered_high_freq))
high_pass_result = np.real(high_pass_result)

# Normalize response to 0-255 for display
high_pass_display = cv2.normalize(
    high_pass_result, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)
print("High-pass mask values:", np.unique(high_pass_mask))
print("High-pass response range:",
      round(float(high_pass_result.min()), 2),
      "to", round(float(high_pass_result.max()), 2))

plt.figure(figsize=(9, 3))
plt.subplot(1, 3, 1); plt.imshow(noisy_image, cmap="gray"); plt.title("Noisy input")
plt.subplot(1, 3, 2); plt.imshow(high_pass_mask, cmap="gray"); plt.title("High-pass mask")
plt.subplot(1, 3, 3); plt.imshow(high_pass_display, cmap="gray"); plt.title("High-pass response")
for a in plt.gcf().axes[:3]: a.axis("off")
plt.tight_layout(); plt.show()
```

Output

High-pass mask values: [0 1]
High-pass response range: -155.53 to 187.31



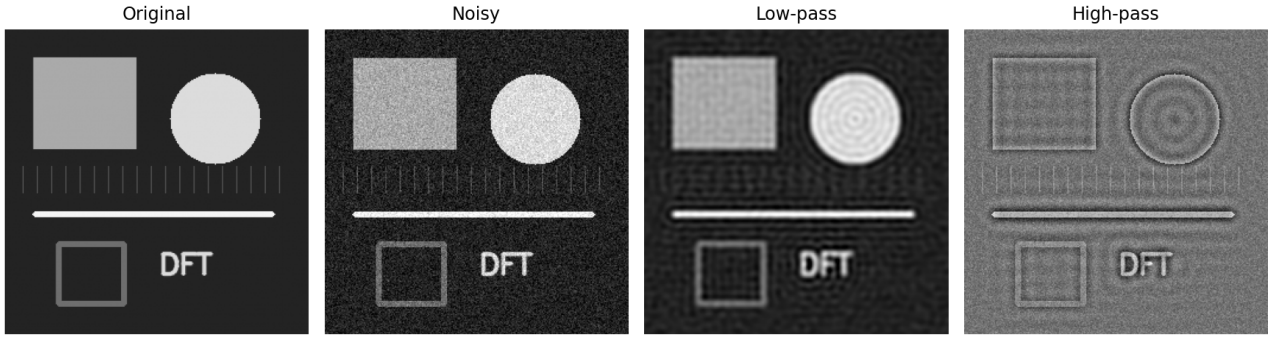
5. Compare the Results and Record Observations

```
print("Original image:", image.shape)
print("Noisy image standard deviation:", round(float(noisy_image.std()), 2))
print("Low-pass image standard deviation:", round(float(low_pass_result.std()), 2))
print("High-pass response standard deviation:", round(float(high_pass_result.std()), 2))

plt.figure(figsize=(10, 3))
for i, (arr, title) in enumerate([
    (image, "Original"), (noisy_image, "Noisy"),
    (low_pass_result, "Low-pass"), (high_pass_display, "High-pass")
]):
    plt.subplot(1, 4, i + 1)
    plt.imshow(arr, cmap="gray"); plt.title(title); plt.axis("off")
plt.tight_layout(); plt.show()
```

Output

Original image: (256, 256)
Noisy image standard deviation: 66.54
Low-pass image standard deviation: 62.69
High-pass response standard deviation: 29.25



Observations

• The centered magnitude spectrum shows where image energy is concentrated in frequency space.
• Low-pass filtering smooths rapid intensity changes and reduces some noise, while also softening edges.
• High-pass filtering emphasizes boundaries and fine details, but can also amplify noise.
• Filter radius controls the trade-off between smoothing and detail preservation. Results depend on the input image and chosen cutoff.

6. Questions and Answers

1. What is the Fourier Transform? Why is it important in digital image processing?

The Fourier Transform represents an image as a combination of spatial-frequency components. It is important because it makes frequency content visible and allows operations such as smoothing, sharpening, periodic-noise suppression, and image restoration to be performed by selectively modifying those components.

2. Differentiate between the spatial domain and the frequency domain.

In the spatial domain, an image is represented by pixel intensities at image coordinates, and filters operate directly on pixels and their neighborhoods. In the frequency domain, the image is represented by frequency coefficients that describe how rapidly intensity changes across space; filtering modifies these coefficients.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

The DFT computes the frequency representation of a discrete image. It supports analysis of periodic patterns and efficient frequency-domain filtering. In practice, the Fast Fourier Transform (FFT) is an algorithm commonly used to calculate the DFT efficiently.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

In the standard FFT output, the zero-frequency component is located at a corner. Shifting it to the center makes the spectrum easier to inspect and makes it convenient to define circular low-pass and high-pass masks around the origin.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

A low-pass filter retains low frequencies and attenuates high frequencies, producing smoothing and often reducing fine noise; applications include denoising and blur/smoothing. A high-pass filter attenuates low frequencies and retains high frequencies, emphasizing edges and fine details; applications include edge enhancement and sharpening. High-pass filters can also emphasize noise.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

After frequency coefficients have been modified, the inverse DFT converts them back into a spatial-domain image. The real component is generally used for display because small imaginary residuals can arise from numerical computation.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

It allows frequency bands or patterns to be targeted directly. It is especially useful when noise or image structures occupy identifiable frequency ranges, and FFT-based methods can be efficient for large images or large convolution kernels. It is not universally better than spatial filtering; the choice depends on the task.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Examples include medical image enhancement, satellite and remote-sensing image analysis, periodic-noise removal in photographs, and image restoration/deblurring. It is also used in texture analysis and feature extraction.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Spatial filters operate directly on pixels and are often simpler and efficient for small kernels such as 3×3 edge detectors or median filters. Frequency filtering typically requires forward and inverse transforms, but can be advantageous for large-kernel convolution or selective frequency suppression. Practical selection depends on image size, filter size, boundary handling, and implementation.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

It can suppress unwanted frequency components, reduce certain noise patterns, and enhance high-frequency structures such as edges. This may improve visibility of features and the quality of restored images. Filter design must be chosen carefully because aggressive filtering can remove useful details or amplify noise.

Result / Conclusion

Frequency-domain image filtering was implemented using the 2-D Discrete Fourier Transform and inverse transform. The centered magnitude spectrum was visualized, and circular low-pass and high-pass masks were applied to reconstruct filtered images. The low-pass output demonstrated smoothing and noise reduction with some loss of detail; the high-pass response emphasized edges and fine structures while also retaining noise. The experiment illustrates how frequency selection affects image enhancement and feature preservation.

Note: The displayed numerical results are generated using the reproducible synthetic test image and fixed random seed shown in the code. Different input images, noise, library versions, or cutoff radii may produce different outputs.